Part 1

First, I split the training set, using the first 90% of the rows as the training set and the last 10% as the test set. For both the training and test sets, I separated the left and right sentences in each row, assigning a label of 1 (correct) or 0 (incorrect) accordingly. Then, I randomly shuffled the sentences in the training set to prepare them for model training.

I initially tried feeding the model sentence pairs and letting it learn the differences, but the model performed worse than when I simply fed it all sentences individually. For the validation set, I kept the paired order because, in the final test, the classifier needs to identify which sentence in each pair has a higher probability of being correct.

To enable the model to process the text numerically, I vectorized the sentences using TF-IDF. Since each sentence is relatively short, I chose to split them into characters rather than words. In addition, TF-IDF can only capture the frequency of characters rather than their order, so I used n-grams to preserve character sequences of lengths 1 to 4 (targeting affixes and subwords). I also removed features that appeared only once to reduce noise, while retaining case sensitivity.

For model selection, I chose logistic regression, a classic binary classification approach. Since the vectors generated by TF-IDF are sparse, I used the *saga* solver for optimization. Then, I applied grid search to find the optimal model parameters.

For evaluation, I had the model make predictions on paired sentences in the validation set, labeling the sentence with the higher probability of y = 1 as 1 and the other as 0. This achieved an accuracy of **0.885**.

Part 2

In Part 2, I needed to corrupt the correct sentences in the training set and challenge the classifier. Therefore, I began by analyzing the weaknesses of the classifier built in Part 1. The classifier is highly sensitive to subwords and affixes (n-grams of length 1–4), but less sensitive to whole words or fixed phrases. Based on this observation, I designed several sentence corruption functions.

First, I implemented the following methods:

1. **frequent_word_swap** – Replace frequently occurring words in the sentence (e.g., “is” → “are”, “do” → “does”). These changes often depend on contextual cues, where the model tends to be weak.
2. **drop_final_s_es** – Remove the final “s” or “es” from words, converting possible plural nouns into singular form. Singular/plural changes depend on word relationships, making them misleading for the model.
3. **plural_swap** – Transform plural forms by swapping “s” and “es” at the end of words.
4. **swap_suffix** – Replace a suffix in a word with another suffix of the same type (e.g., “available” → “availative”), targeting the classifier’s tendency to focus more on suffixes than on the entire word.

5. **swap_suffix_words** – Swap the positions of words with the same type of suffix. Since such positions depend on semantics, they are difficult for the classifier to detect.

For these functions, I ensured that at most one transformation was randomly applied to each sentence, minimizing excessive noise. If none of the functions could be applied (e.g., no valid regex match), a fallback strategy of random word or character swapping was used instead.

For evaluation, I selected the last 10% of the right-hand incorrect sentences from *train.txt* and *part2.txt* (excluding any data used to train the model). Now, instead of performing pairwise classification as in Part 1, I focused only on the incorrect sentences and evaluated how well the classifier could identify them as incorrect. On the original sentences, the model achieved an accuracy of **0.5993**, whereas on the corrupted sentences, the accuracy dropped sharply to **0.1848**, demonstrating that the classifier was effectively misled.